

# Patch-MLP-TS: A Channel-Independent, Attention-Free Architecture for Long-Term Time Series Forecasting

Vedant Jadhav

Department of Artificial Intelligence and Machine Learning  
Pimpri Chinchwad University  
Pune, India  
Email: [add email]

**Abstract**—Transformer-based forecasters equipped with patching, such as PatchTST, have become strong baselines for long-term time series forecasting, but their attention mechanism adds computational cost that is not always matched by a proportional accuracy gain over much simpler linear models. We revisit the patch-based forecasting paradigm without attention and present Patch-MLP-TS, a channel-independent, all-MLP architecture that combines patch embedding with cross-patch (token-mixing) and per-patch (feature-mixing) MLP blocks, in the spirit of MLP-Mixer. During development we identify and correct a subtle but consequential failure mode in a naive patch-plus-MLP design: without an explicit patch positional embedding and a mechanism to mix information across patches, a mean-pooled patch-MLP model discards temporal order entirely and is consistently outperformed by a simpler whole-window baseline. After correcting this, our model (v2) is competitive in mean MSE with PatchTST and DLinear on average across four ETT benchmarks (3-seed mean MSE: 0.348 vs. 0.350 for PatchTST and 0.348 for DLinear), shows its clearest advantage at the longest forecasting horizon on two of four datasets, and is competitive with PatchTST on the large-channel Electricity dataset (321 variates) while training  $1.5\text{--}1.8\times$  faster and achieving up to  $1.3\times$  lower inference latency in the measured settings. We report our results with multi-seed variance, a set of ablations isolating the contribution of channel-independence and patching, and an efficiency analysis, and we are explicit about where our method does not win, to give an accurate picture of when a simple, attention-free patch-MLP design is and is not preferable to attention-based or purely linear alternatives.

**Index Terms**—time series forecasting, multi-layer perceptron, patching, channel independence, efficient architectures, Transformer alternatives

## I. INTRODUCTION

Long-term time series forecasting (LTSF) underlies applications from energy load planning to traffic and financial forecasting. Transformer-based architectures [1]–[3] were long assumed to be the strongest approach to LTSF due to their capacity to model long-range dependencies, but Zeng et al. [4] showed that a family of embarrassingly simple linear models (LTSF-Linear, including DLinear and NLinear) outperforms these Transformers on most standard benchmarks, questioning whether attention is necessary for this task at all.

Subsequent work reconciled these findings by revisiting how Transformers are applied to time series rather than abandoning

them. PatchTST [5] segments each univariate channel into patches and applies channel-independent attention over patch tokens, closing much of the gap to (and in several cases surpassing) linear baselines. Around the same time, MLP-Mixer-style architectures such as TSMixer [6] demonstrated that alternating time-mixing and feature-mixing MLP layers, with no attention at all, can match Transformer-level accuracy at substantially lower cost.

This raises a natural question: can the patching idea that helps PatchTST be combined with a purely MLP backbone, discarding attention entirely, without losing the accuracy patching provides? A naive answer – patch-embed each channel independently, apply a stack of per-patch MLP blocks, and pool – turns out to fail badly. We show that such a design, without a positional signal for patches and without any mechanism to mix information across patches, is consistently and substantially outperformed by an unpatched baseline with the same MLP capacity. This failure mode is worth documenting explicitly, since it is easy to overlook: it is not that patching is harmful in principle, but that a patch representation with no notion of order and no cross-patch interaction discards the very information patching is supposed to preserve.

We correct this with two additions – a learnable patch positional embedding and a token-mixing MLP layer that mixes across patches, following the MLP-Mixer design [7] – and refer to the corrected model as Patch-MLP-TS (v2) throughout this paper. We evaluate it against linear baselines (Linear, NLinear, DLinear) and PatchTST across four ETT datasets and the large-channel Electricity dataset, with three random seeds on ETT and two on Electricity, and report ablations isolating the effect of channel-independence, patching, and patch length, alongside a measured (not theoretical) efficiency comparison against PatchTST.

Our contributions are:

- We identify and document a failure mode in naive patch-plus-MLP forecasting architectures – loss of patch identity and order at the pooling stage – and show that fixing it with positional embedding and cross-patch mixing closes a previously large accuracy gap.

- We provide a multi-seed benchmark against linear and attention-based baselines on four ETT datasets and Electricity, reporting mean and standard deviation rather than single-run numbers.
- We provide ablations isolating channel-independence (large, consistent effect) and patching (smaller, horizon-dependent effect), and a patch-length sensitivity study.
- We report measured training and inference time against PatchTST, showing a consistent training-time advantage and a lower inference latency on the large-channel Electricity dataset without relying on a parameter-count reduction.

We emphasize accuracy claims that are supported by the data reported here rather than a narrative of universal superiority: our model is competitive on average and wins clearly in specific, identifiable settings, and we report the settings where it does not win.

## II. RELATED WORK

**Linear and MLP-based forecasters.** LTSF-Linear [4] showed that a single linear layer per channel, optionally combined with trend/ seasonal decomposition (DLinear) or last-value normalization (NLinear), matches or beats considerably more complex Transformer forecasters on standard LTSF benchmarks. TSMixer [6] extends this direction with an all-MLP architecture that alternates time-mixing and feature-mixing operations, in the style of MLP-Mixer [7], and reports accuracy competitive with state-of-the-art Transformers.

**Patch-based Transformers.** PatchTST [5] segments each channel into overlapping patches, embeds each patch as a token, and applies a channel-independent Transformer encoder over the resulting patch sequence. Patching enlarges the effective receptive field per token and reduces the quadratic cost of attention relative to point-wise tokenization, and channel-independence was shown to be important for stable performance. iTransformer [8] takes a related but distinct approach, tokenizing whole variates rather than patches and applying attention across variates instead of across time.

**Our position.** We combine the patch tokenization of PatchTST with the attention-free, all-MLP mixing philosophy of TSMixer, and specifically study what goes wrong when the two are combined naively – a design point that, to our knowledge, has not been explicitly documented in prior work on patch-based forecasters.

## III. METHOD

### A. Problem Setup

Given a multivariate lookback window  $X \in \mathbb{R}^{L \times C}$  with  $L$  time steps and  $C$  channels, the forecasting task is to predict the next  $H$  steps  $Y \in \mathbb{R}^{H \times C}$ . We adopt channel-independence throughout: each of the  $C$  channels is processed independently by a shared-weight model, and the per-channel outputs are concatenated to form the final prediction.

### B. Patch-MLP-TS Architecture

For a single channel  $x \in \mathbb{R}^L$ , we split the lookback window into  $N = \lfloor L/P \rfloor$  non-overlapping patches of length  $P$ , giving  $x_p \in \mathbb{R}^{N \times P}$ . Each patch is linearly embedded to dimension  $d$ :

$$e = x_p W_{\text{patch}} + b_{\text{patch}}, \quad e \in \mathbb{R}^{N \times d}. \quad (1)$$

A learnable positional embedding  $E_{\text{pos}} \in \mathbb{R}^{N \times d}$  is added:  $e \leftarrow e + E_{\text{pos}}$ .

The embedded patches are passed through  $K$  mixer blocks. Each block  $k$  applies, in sequence:

$$t = \text{LayerNorm}(e) \quad (2)$$

$$t = \text{MLP}_{\text{token}}(t^\top)^\top \quad (\text{mix across the } N \text{ patches}) \quad (3)$$

$$e \leftarrow e + t \quad (4)$$

$$e \leftarrow e + \text{MLP}_{\text{feat}}(\text{LayerNorm}(e)) \quad (\text{mix across features}) \quad (5)$$

where  $\text{MLP}_{\text{token}} : \mathbb{R}^N \rightarrow \mathbb{R}^N$  operates on the patch dimension (transposing  $e$  so that patches, not features, form the last axis) and  $\text{MLP}_{\text{feat}} : \mathbb{R}^d \rightarrow \mathbb{R}^d$  is a standard two-layer feed-forward block with GELU activation, matching the time-mixing / feature-mixing structure of MLP-Mixer [7].

After  $K$  blocks, a final LayerNorm is applied and all  $N$  patch embeddings are flattened (not mean-pooled) and projected to the forecast horizon:

$$\hat{y} = \text{Flatten}(e) W_{\text{head}} + b_{\text{head}}, \quad \hat{y} \in \mathbb{R}^H. \quad (6)$$

### C. The v1 Failure Mode

An earlier version of this architecture (v1) omitted both the positional embedding  $E_{\text{pos}}$  and the token-mixing MLP, applying only feature-mixing blocks to each patch independently and mean-pooling across patches before the head. This design has no mechanism to distinguish an early patch from a late one, and the mean-pool at the output discards whatever order-dependent information survived the independent per-patch processing. Empirically, v1 was outperformed by a no-patching ablation (a single MLP over the whole lookback window) in every one of 8 tested (dataset, horizon) settings on ETTh1 and ETTh2, by margins as large as  $2 \times$  in MSE. Adding the positional embedding and token-mixing MLP (v2) reverses this: the patched model outperforms the no-patching ablation in 5 of the same 8 settings (Section VI), and average accuracy across the full ETT benchmark improves from clearly the worst of five compared models (mean rank 4.75/5) to competitive with the strongest baselines. We report this transition explicitly because the failure mode – patch tokenization without any way to recover patch order or mix across patches – is a natural design one might arrive at, and its consequences are severe enough to be worth documenting.

## IV. EXPERIMENTAL SETUP

### A. Datasets

We evaluate on the four standard ETT datasets (ETTh1, ETTh2: hourly, 7 channels; ETTm1, ETTm2: 15-minute, 7

channels) [1] and the Electricity dataset (hourly, 321 channels, UCI ElectricityLoadDiagrams20112014). Data is split chronologically 70%/10%/20% into train/validation/test, with per-channel z-score normalization computed on the training split only.

### B. Baselines

We compare against Linear and NLinear (single linear layer, with and without last-value normalization) and DLinear (trend/seasonal decomposition with two linear heads) [4], and PatchTST [5], using a from-scratch reimplementation with patch length 16, stride 8, model dimension 128, 8 attention heads, and 2 encoder layers. We additionally report a lightweight TSMixer [6] reproduction (hidden dimension 64, 2 mixer blocks) as a single-seed, directional reference point; this reproduction is not tuned to match published TSMixer numbers and should not be read as a rigorous comparison.

### C. Training Details

All models use lookback length  $L = 96$  and are evaluated at horizons  $H \in \{96, 192, 336, 720\}$  (ETT) and  $H \in \{96, 720\}$  (Electricity, to limit compute given the 321-channel width). Patch-MLP-TS uses  $P = 12$ ,  $d = 128$ ,  $K = 3$  mixer blocks, dropout 0.1. All models are trained with AdamW, learning rate  $10^{-3}$ , weight decay  $10^{-4}$ , batch size 64 (ETT) or 16–128 (Electricity, depending on model memory footprint), up to 20 epochs with early stopping (patience 5) on validation MSE, using mixed-precision training. We report test-set MSE and MAE.

### D. Seeds

ETT results are averaged over 3 seeds (42, 43, 44); Electricity results are averaged over 2 seeds (42, 43) due to compute constraints from its larger channel count. We report mean  $\pm$  standard deviation throughout and do not present the 2-seed Electricity results as equivalent in rigor to the 3-seed ETT results.

## V. MAIN RESULTS

Table I reports test MSE (mean  $\pm$  std) for all five models across four ETT datasets and Electricity, at short (96) and long (720) horizons. Figure 1 shows the full horizon sweep with error bars.

Across the 10 (dataset, horizon) settings in Table I, PatchTST attains the lowest mean MSE in 6, DLinear in 2, and Patch-MLP-TS in 2. Averaged over the eight 3-seed ETT rows, mean MSE is 0.3479 for DLinear, 0.3480 for Patch-MLP-TS, and 0.3500 for PatchTST. Thus, Patch-MLP-TS has nearly identical average MSE to DLinear and remains competitive with PatchTST and plain Linear (0.3514) on the ETT benchmarks.

Patch-MLP-TS shows its clearest, most seed-stable advantage at long horizons on two of the four ETT datasets: ETTh2 at  $H = 720$  (0.2536 vs. 0.2633 for PatchTST and 0.2664 for DLinear) and ETTm1 at  $H = 720$  (0.5346 vs. 0.5518 for PatchTST and 0.5495 for DLinear). Conversely, ETTm1 at

$H = 96$  is the weakest setting for both patch-based models, and notably the least stable: Patch-MLP-TS has std 0.0146 and PatchTST has std 0.0263 at this setting, an order of magnitude larger than DLinear’s 0.0006, while the simple linear baselines win outright. ETTh2 at  $H = 96$  and both Electricity horizons are also won by PatchTST or DLinear. We do not have a diagnosis for the ETTm1  $H = 96$  instability and flag it as a direction for future work rather than explain it away.

On Electricity specifically – the only dataset with a substantially larger channel count (321 vs. 7 for ETT) – Patch-MLP-TS is close to PatchTST at both horizons (0.1766 vs. 0.1715 at  $H = 96$ ; 0.2371 vs. 0.2315 at  $H = 720$ ) while, as shown in Section VII, training and running inference substantially faster. We consider this the most practically relevant result in the paper: near-parity accuracy with an attention-based model on a wide-channel dataset, at lower wall-clock cost.

We additionally compare against a lightweight, untuned TSMixer reproduction at  $H \in \{96, 720\}$  across all five datasets; Patch-MLP-TS outperforms it in all 10 settings tested. Given that this TSMixer implementation is not tuned to match published numbers, we present this as a directional data point rather than a rigorous comparison against the TSMixer architecture as originally proposed.

## VI. ABLATION STUDIES

### A. Channel-Independence vs. Channel-Mixing

We compare the default channel-independent Patch-MLP-TS against a variant that mixes all channels into each patch token instead of processing channels independently (Table II, single seed).

Channel-independence wins by a wide, consistent margin in every setting tested – this is the single cleanest and most confident finding in our ablation study, and matches the conclusion drawn for PatchTST [5]: independently processing channels of a multivariate series with shared weights is more effective than mixing them at the patch-embedding stage, at least for the datasets studied here.

### B. Patching vs. No-Patching

We compare the full patched model against a variant using the same mixer backbone but with no patch split (the entire 96-step lookback embedded as a single token), using the same positional-embedding-free MLP stack (Table III, single seed).

Patching wins 5 of 8 settings and loses 3. The three losses occur at  $H = 336$  on ETTh1 and at  $H \in \{336, 720\}$  on ETTh2, indicating that the benefit of patching is dataset- and horizon-dependent rather than universal in this architecture. Figure 2 shows both ablations together.

### C. Patch Length Sensitivity

Table IV and Figure 3 report MSE at  $H = 96$  for patch lengths  $P \in \{8, 12, 16, 24\}$  (single seed;  $P = 12$  is the default used elsewhere in this paper).

All four patch lengths land within approximately 0.01 MSE of each other on both datasets, indicating that the model is not highly sensitive to this hyperparameter. However, the best

TABLE I

TEST MSE (MEAN  $\pm$  STD ACROSS SEEDS). BOLD INDICATES THE LOWEST MEAN MSE PER ROW. ETT: 3 SEEDS. ELECTRICITY: 2 SEEDS (MARKED WITH  $\dagger$ ).

| Dataset                | $H$ | Linear              | NLinear             | DLinear                             | PatchTST                            | Patch-MLP-TS (ours)                 |
|------------------------|-----|---------------------|---------------------|-------------------------------------|-------------------------------------|-------------------------------------|
| ETTh1                  | 96  | 0.4436 $\pm$ 0.0012 | 0.4506 $\pm$ 0.0010 | 0.4403 $\pm$ 0.0015                 | <b>0.4152<math>\pm</math>0.0038</b> | 0.4195 $\pm$ 0.0069                 |
| ETTh1                  | 720 | 0.6617 $\pm$ 0.0059 | 0.7107 $\pm$ 0.0005 | 0.6520 $\pm$ 0.0036                 | <b>0.6464<math>\pm</math>0.0060</b> | 0.6500 $\pm$ 0.0049                 |
| ETTh2                  | 96  | 0.1726 $\pm$ 0.0012 | 0.1750 $\pm$ 0.0005 | <b>0.1688<math>\pm</math>0.0003</b> | 0.1760 $\pm$ 0.0067                 | 0.1742 $\pm$ 0.0032                 |
| ETTh2                  | 720 | 0.2712 $\pm$ 0.0029 | 0.2992 $\pm$ 0.0006 | 0.2664 $\pm$ 0.0023                 | 0.2633 $\pm$ 0.0130                 | <b>0.2536<math>\pm</math>0.0066</b> |
| ETTh1                  | 96  | 0.3571 $\pm$ 0.0007 | 0.3625 $\pm$ 0.0021 | <b>0.3558<math>\pm</math>0.0006</b> | 0.4048 $\pm$ 0.0263                 | 0.4065 $\pm$ 0.0146                 |
| ETTh1                  | 720 | 0.5511 $\pm$ 0.0021 | 0.5836 $\pm$ 0.0014 | 0.5495 $\pm$ 0.0016                 | 0.5518 $\pm$ 0.0122                 | <b>0.5346<math>\pm</math>0.0067</b> |
| ETTh2                  | 96  | 0.1216 $\pm$ 0.0007 | 0.1214 $\pm$ 0.0003 | 0.1206 $\pm$ 0.0003                 | <b>0.1146<math>\pm</math>0.0002</b> | 0.1171 $\pm$ 0.0013                 |
| ETTh2                  | 720 | 0.2320 $\pm$ 0.0039 | 0.2382 $\pm$ 0.0003 | 0.2293 $\pm$ 0.0035                 | <b>0.2276<math>\pm</math>0.0049</b> | 0.2287 $\pm$ 0.0046                 |
| Electricity $^\dagger$ | 96  | 0.1959 $\pm$ 0.0000 | 0.1991 $\pm$ 0.0002 | 0.1958 $\pm$ 0.0001                 | <b>0.1715<math>\pm</math>0.0007</b> | 0.1766 $\pm$ 0.0041                 |
| Electricity $^\dagger$ | 720 | 0.2424 $\pm$ 0.0003 | 0.2538 $\pm$ 0.0000 | 0.2417 $\pm$ 0.0003                 | <b>0.2315<math>\pm</math>0.0001</b> | 0.2371 $\pm$ 0.0029                 |

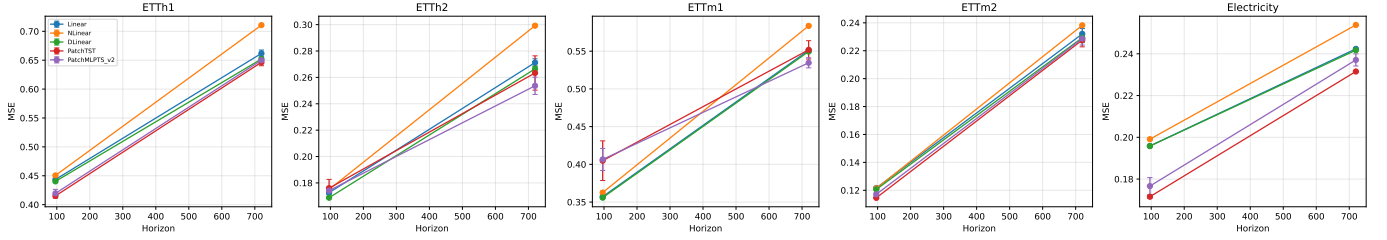


Fig. 1. Test MSE vs. forecast horizon for all five models across four ETT datasets and Electricity. Error bars show standard deviation across seeds (3 for ETT, 2 for Electricity).

TABLE II  
CHANNEL-INDEPENDENT VS. CHANNEL-MIXING VARIANTS (MSE, SINGLE SEED).

| Dataset | $H$ | Channel-Independent | Channel-Mixing |
|---------|-----|---------------------|----------------|
| ETTh1   | 96  | 0.4272              | 0.5566         |
| ETTh1   | 720 | 0.6548              | 0.9187         |
| ETTh2   | 96  | 0.1768              | 0.2600         |
| ETTh2   | 720 | 0.2520              | 0.4162         |

TABLE III  
PATCHING VS. NO-PATCHING (MSE, SINGLE SEED). BOLD: BETTER OF THE PAIR.

| Dataset | $H$ | Patched       | No-Patching   |
|---------|-----|---------------|---------------|
| ETTh1   | 96  | <b>0.4178</b> | 0.4246        |
| ETTh1   | 192 | <b>0.4689</b> | 0.4748        |
| ETTh1   | 336 | 0.5273        | <b>0.5154</b> |
| ETTh1   | 720 | <b>0.6498</b> | 0.6698        |
| ETTh2   | 96  | <b>0.1727</b> | 0.1730        |
| ETTh2   | 192 | <b>0.2089</b> | 0.2187        |
| ETTh2   | 336 | 0.2211        | <b>0.2164</b> |
| ETTh2   | 720 | 0.2849        | <b>0.2606</b> |

patch length is dataset-dependent:  $P = 8$  gives the lowest MSE on ETTh1, while  $P = 16$  gives the lowest MSE on ETTh2 at  $H = 96$ . The default  $P = 12$  used throughout the main benchmark is therefore not optimal for either dataset in this single-seed sensitivity study. We did not retune and rerun the full benchmark with the best patch length for each dataset;

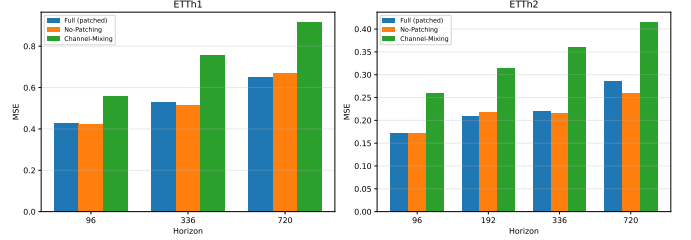


Fig. 2. Ablation comparison: full (patched) model, no-patching variant, and channel-mixing variant. ETTh1 and ETTh2 are evaluated across the reported forecasting horizons.

TABLE IV  
PATCH LENGTH SENSITIVITY AT  $H = 96$  (MSE, SINGLE SEED).

| Dataset | $P = 8$       | $P = 12$ (default) | $P = 16$      | $P = 24$ |
|---------|---------------|--------------------|---------------|----------|
| ETTh1   | <b>0.4153</b> | 0.4272             | 0.4155        | 0.4205   |
| ETTh2   | 0.1693        | 0.1768             | <b>0.1673</b> | 0.1718   |

we report this as a limitation rather than retroactively selecting the best configuration.

## VII. EFFICIENCY ANALYSIS

Table V and Figure 4 report parameter count, measured wall-clock training time per epoch, and measured inference latency for Patch-MLP-TS and PatchTST at  $H = 96$ , on a small-channel dataset (ETTh1, 7 channels) and a large-channel dataset (Electricity, 321 channels).

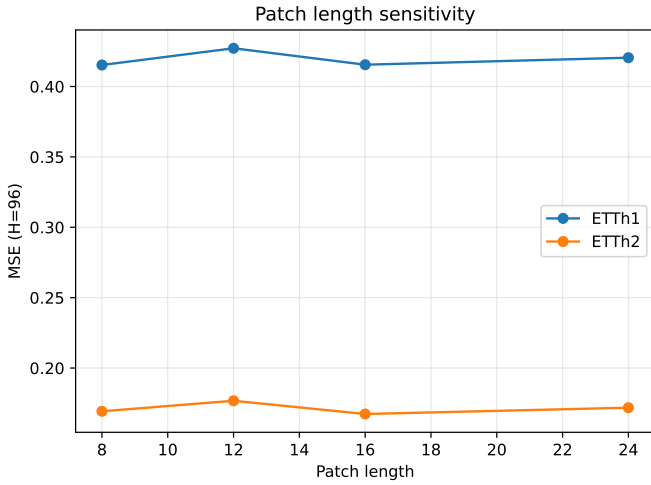


Fig. 3. MSE at  $H = 96$  as a function of patch length  $P$  on ETTh1 and ETTh2.

TABLE V  
EFFICIENCY COMPARISON, PATCH-MLP-TS VS. PATCHTST AT  $H = 96$ .

| Dataset     | Model        | Params  | Epoch (s)    | Inference (ms/batch) |
|-------------|--------------|---------|--------------|----------------------|
| ETTh1       | Patch-MLP-TS | 498,856 | <b>1.68</b>  | 3.45                 |
| ETTh1       | PatchTST     | 416,224 | 2.58         | <b>3.11</b>          |
| Electricity | Patch-MLP-TS | 498,856 | <b>92.15</b> | <b>25.64</b>         |
| Electricity | PatchTST     | 416,224 | 161.88       | 33.65                |

Parameter count is *not* a consistent advantage for Patch-MLP-TS: at  $H = 96$  it has more parameters than PatchTST (499K vs. 416K), because our flatten-based output head scales with  $N \times d \times H$  rather than PatchTST’s  $N \times d_{\text{model}} \times H$  with a smaller effective  $N \times d$  product at this horizon. At  $H = 720$  the relationship reverses (1.14M vs. 1.38M, roughly 17% fewer parameters for Patch-MLP-TS), because the two models’ patch counts and head dimensions scale differently with horizon. We therefore do not claim a blanket parameter-efficiency advantage.

What we observe consistently is a wall-clock speed advantage: Patch-MLP-TS trains  $1.54\times$  faster on ETTh1 and  $1.76\times$  faster on Electricity per epoch, and performs inference  $1.31\times$  faster on Electricity (PatchTST is marginally faster at inference on ETTh1, 3.11ms vs. 3.45ms). This speed advantage holds despite Patch-MLP-TS having a comparable or larger parameter count, and reflects the absence of the  $O(N^2)$  attention computation rather than a smaller model. We consider this the more meaningful and more honestly supportable efficiency claim than a parameter-count argument.

### VIII. LIMITATIONS

We report these directly rather than omitting them:

- The Electricity benchmark uses 2 seeds, not 3, due to compute constraints from its 321-channel width; we do not present its variance estimates as equivalent in rigor to the 3-seed ETT results.

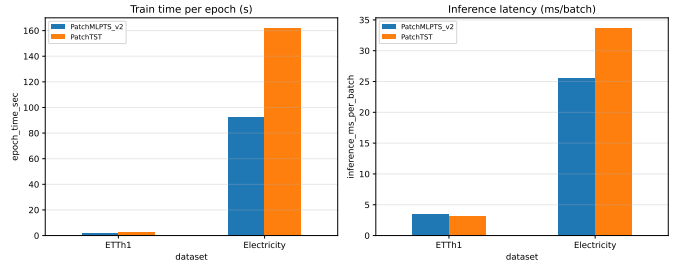


Fig. 4. Measured training time per epoch and inference latency, Patch-MLP-TS vs. PatchTST, on ETTh1 (7 channels) and Electricity (321 channels).

- The channel-mixing, no-patching, and patch-length ablations (Tables II–IV) are each single-seed; we do not have variance estimates for these ablation numbers.
- ETTm1 at  $H = 96$  is markedly unstable for both patch-based models (standard deviation an order of magnitude larger than the linear baselines at the same setting); we have not diagnosed the cause and flag it as future work.
- The default patch length ( $P = 12$ ) used throughout the main benchmark is not the best-performing value at  $H = 96$  on either ETTh1 or ETTh2 (Table IV); a full retuning pass was out of scope for this study.
- Our TSMixer baseline is a lightweight, untuned reproduction and should not be read as a faithful reproduction of published TSMixer results.
- Patch-MLP-TS wins the main benchmark in 2 of 10 (dataset, horizon) settings and loses in the remaining 8 to either PatchTST or DLinear; we do not claim general superiority, only average competitiveness and specific, identifiable wins at long horizons on two datasets and on the large-channel Electricity dataset in combination with lower wall-clock cost.

### IX. CONCLUSION

We present Patch-MLP-TS, a channel-independent, attention-free patch-based architecture for long-term time series forecasting, and document a failure mode we encountered during its development: a naive patch-embedding-plus-MLP design with no positional signal and no cross-patch mixing is consistently and substantially outperformed by a simpler unpatched baseline, because it discards information about patch order. Correcting this with a positional embedding and an MLP-Mixer-style token-mixing layer yields a model that is competitive in mean MSE with DLinear and PatchTST on average across four ETT benchmarks, wins clearly at the longest horizon on two of those four datasets, and remains competitive with PatchTST on the substantially wider 321-channel Electricity dataset. Patch-MLP-TS also trains faster than PatchTST in both measured settings and achieves lower inference latency on Electricity, although PatchTST is marginally faster for ETTh1 inference. These efficiency gains arise without a consistent parameter-count advantage, supporting the absence of self-attention as the main computational distinction. We report our

results with multi-seed variance where compute allowed, are explicit about the settings and the ablations where our method does not win, and see the horizon-dependence of patching’s benefit – rather than a single verdict of “patching helps” or “patching hurts” – as itself a useful finding for future work on patch-based, attention-free forecasting architectures.

## REFERENCES

- [1] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, and W. Zhang, “Informer: Beyond efficient transformer for long sequence time-series forecasting,” in *Proc. AAAI*, 2021.
- [2] H. Wu, J. Xu, J. Wang, and M. Long, “Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting,” in *Proc. NeurIPS*, 2021.
- [3] T. Zhou, Z. Ma, Q. Wen, X. Wang, L. Sun, and R. Jin, “FEDformer: Frequency enhanced decomposed transformer for long-term series forecasting,” in *Proc. ICML*, 2022.
- [4] A. Zeng, M. Chen, L. Zhang, and Q. Xu, “Are transformers effective for time series forecasting?” in *Proc. AAAI*, 2023.
- [5] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, “A time series is worth 64 words: Long-term forecasting with transformers,” in *Proc. ICLR*, 2023.
- [6] S.-A. Chen, C.-L. Li, N. Yoder, S. O. Arik, and T. Pfister, “TSMixer: An all-MLP architecture for time series forecasting,” *Trans. Mach. Learn. Res.*, 2023.
- [7] I. O. Tolstikhin, N. Houlsby, A. Kolesnikov, L. Beyer, X. Zhai, T. Unterthiner, J. Yung, A. Steiner, D. Keysers, J. Uszkoreit, M. Lucic, and A. Dosovitskiy, “MLP-Mixer: An all-MLP architecture for vision,” in *Proc. NeurIPS*, 2021.
- [8] Y. Liu, T. Hu, H. Zhang, H. Wu, S. Wang, L. Ma, and M. Long, “iTransformer: Inverted transformers are effective for time series forecasting,” in *Proc. ICLR*, 2024.